

Deep Dive — Cost, Protocols, Failure Modes, Control, and the Decision Framework

Part 07 —  Agentic AI Systems · 07-agentic-ai/notes/02-coordination-control-and-evaluation.md

Notes/01 gave you the shapes. This note gives you the arithmetic and the judgment calls: why multi-agent systems cost what they cost, how agents actually talk to each other, the specific ways coordination breaks, how much control you give up as autonomy rises, and — the part you'll actually use — a framework for deciding whether a request in front of you needs any of this at all.

1. Why multi-agent is expensive, and often unnecessary

The token-cost multiplication

Every agent in a multi-agent system carries its own context: system prompt, task description, tool definitions, and a growing transcript. None of that is shared for free — if agent B needs to know what agent A learned, A has to *say* it, in tokens, and B has to *read* it, in tokens. Coordination is not overhead layered on top of the "real" work; in a multi-agent system, coordination *is* a large fraction of the real work.

Anthropic's own engineering write-up on the multi-agent research system behind Claude puts a number on this: **"agents typically use about 4x more tokens than chat interactions, and multi-agent systems use about 15x more tokens than chats."** That's not a inefficiency to be optimized away — it's structural. A supervisor reading and summarizing three workers' outputs, each of which read and summarized their own tool calls, is doing real work with real tokens at every layer.

15x is a lot. For it to be worth paying, the *value* of the task has to scale with it — which is exactly why Anthropic's own use case (open-ended research requiring breadth) justifies the cost and a customer support bot almost never will.

Context isolation: the actual benefit you're paying for

The reason to accept that multiplier is **context isolation**. A single agent's context window is a shared, ever-growing resource — every tool result, every intermediate thought, every retrieved document competes for the same attention budget, and irrelevant context measurably degrades a model's performance on the parts that matter (this is "context rot," covered in Module 04). Splitting work across agents gives each one a clean, scoped context containing only what its sub-task needs. A research sub-agent chasing one line of inquiry doesn't have to carry the other four lines of inquiry in its context; it only has to carry its own.

That's the trade in one sentence: **you're paying tokens to buy attention**. Whether that's a good trade depends entirely on whether the single agent's context was actually the bottleneck.

Hand-off information loss: the cost you don't see coming

Context isolation has a mirror-image cost. If agent A's work has to influence agent B's decision, and A and B don't share a context window, the *only* channel between them is whatever A chooses to write down. Every hand-off is a compression step, and compression is lossy by construction — the summary is, definitionally, smaller than what generated it. `code/handoff_information_loss.py` makes this concrete and measurable rather than asserting it: an agent with a rich internal trace produces a lossy hand-off summary, and a downstream agent's decision quality degrades in direct proportion to what the summary dropped. Read that script's output before you take the "just add another agent" instinct at face value.

The honest position

Most multi-agent systems should be one agent with better tools. LangChain's own guidance says this plainly, and it's worth repeating because the industry's marketing pulls the other way: a single agent with a well-scoped tool list and a decent prompt gets you most of what a naive multi-agent split gets you, at a fraction of the cost and with a debuggable, linear trace. If you're building a multi-agent system because the org chart has three teams, or because it demos better, or because "multi-agent" is the exciting phrase this quarter — stop. Read Cognition's "Don't Build Multi-Agents" for the sharpest version of this argument.

That said, the counter-cases are real, and the same Anthropic write-up that documents the 15x cost also documents why they paid it. Reach for multiple agents when **at least one** of these is genuinely true:

Real reason to split	What it looks like
Independent, parallelizable sub-tasks	Five unrelated research threads that don't depend on each other's results — the fan-out finishes in the time of the <i>slowest</i> branch, not the sum
Different permission or data-residency needs per role	An agent with access to PII/financial systems must not be the same context as one calling public search
A tool list too long for one model to select from reliably	Past roughly a few dozen tools, selection accuracy drops; splitting by domain (billing tools vs. shipping tools) fixes what a longer prompt can't
Genuinely different models or cost tiers per sub-task	A cheap, fast model triages; an expensive, careful model handles the 5% that needs it

Everything else — "it feels more sophisticated," "we have separate teams," "the framework makes it easy" — is not a reason. `code/topology_simulator.py` runs the same toy task through all five topologies and a single-agent baseline and prints the token/latency/success trade-off as numbers, not vibes, so you can see exactly where the crossover point is.

2. Communication protocols between agents

However agents are arranged, they need a channel. Three patterns cover almost everything:

Pattern	Mechanism	Feels like	Where it shows up
Shared scratchpad / blackboard	All agents read and write one shared document/memory; no addressing, whoever's turn it is reads the current state	A whiteboard everyone can walk up to	Early multi-agent research systems; simple crew setups where "notes so far" is one shared string
Structured message passing	Agents send each other typed messages (task, result, request) with an explicit sender/recipient; no shared memory	Email between coworkers who don't share a desk	Peer hand-off and hierarchical topologies; each hop is a discrete, loggable message
Shared state (channels)	A structured, typed state object that every agent reads and writes specific fields of, under a coordinator's control	A shared database with column-level permissions	This is <i>literally</i> what a graph framework's state channels are — LangGraph's <code>State TypedDict</code> , updated by reducers, is shared state with schema discipline (Module 16)

None of these is strictly better — they trade off differently. Blackboard is simplest to build and worst to reason about (anyone can see and touch anything, so cause-and-effect gets murky at scale). Message passing is the most auditable (every hand-off is a discrete artifact you can log and replay) but pays a serialization cost at every hop. Shared state gives you the least redundant communication (agents write once, others read directly) but requires you to design a schema up front — which is exactly the discipline a graph-based framework forces on you.

A2A vs. MCP — connect this to Module 09

It's easy to conflate these because both are "open protocols for agentic systems" announced around the same period, but they solve different problems:

- **MCP (Model Context Protocol)** — connects an agent to **tools and data**. It's the standardized way a model gets a list of callable functions, a resource to read, or a prompt template, regardless of which vendor built the tool server. Covered in depth in Module 09 [09-mcp]. The relationship is agent → tool/data.
- **A2A (Agent2Agent protocol)** — connects an agent to **another agent**. Google announced it in 2025 (now under Linux Foundation stewardship); it defines Agent Cards (how an agent advertises its capabilities), Tasks (the unit of delegated work), and a transport (HTTP, Server-Sent Events, JSON-RPC) for one agent to discover, delegate to, and get results back from another — potentially built by a different vendor, on a different framework, with its own internal architecture the caller never sees. The relationship is agent → agent.

Put simply: **MCP is how an agent gets a screwdriver; A2A is how an agent asks another agent to build the whole shelf**. A supervisor topology built with A2A treats each worker as an opaque black box with a published interface, rather than a subgraph you control and can see inside — useful for cross-organization agent ecosystems, overkill for a multi-agent system you own end to end (there, plain function calls or a framework's native hand-off primitive are simpler and cheaper). Read the spec at a conceptual level; implementation is out of scope here.

3. Coordination failure modes unique to multi-agent

Single-agent systems fail in ways Module 06 already covered (bad tool call, hallucinated argument, infinite retry loop). Multi-agent systems fail in those ways *plus* an additional layer that only exists because more than one agent is involved:

- **Deadlock / livelock between hand-off agents.** Two agents each decide the task belongs to the other (§2.2 of notes/01). Deadlock: both wait for the other to act, nothing progresses. Livelock: both keep acting, but their actions cancel out — the peer-to-peer topology has no arbiter to break the tie unless you add one (a hop counter, a "this hand-off has already happened once, escalate to human" rule).
- **Information loss in summarized hand-offs.** Covered viscerally in `code/handoff_information_loss.py`. The generalized version: any time context isn't shared verbatim, whoever wrote the summary decided — possibly wrongly — what the next agent needs to know.
- **One agent's hallucination poisoning downstream agents.** In a pipeline or supervisor topology, a downstream agent generally *trusts* the upstream agent's output as ground truth — it has no way to independently verify "agent 2 said the invoice total is \$4,200" unless you build in a verification step. A hallucinated fact from stage 1 becomes a confidently-stated fact by stage 4, with no record that it was ever uncertain. This is strictly worse than a single agent hallucinating, because a single agent's hallucination is visible in one trace; a poisoned hand-off launders it through multiple agents' apparent agreement, which reads as corroboration to a human reviewer.
- **Redundant work.** Without explicit task-claiming or a supervisor tracking what's been assigned, two workers can independently decide to handle the same sub-task — wasting tokens and, worse, sometimes producing two different answers to the same question with no signal about which is more trustworthy.
- **Disagreement / thrashing in debate patterns.** Covered in §2.4 of notes/01: without an explicit convergence rule, two confident agents holding opposite positions will spend rounds re-litigating instead of updating toward each other.

The common thread: **all five of these are invisible to a per-agent eval**. Each individual agent can pass its own unit test and the *system* can still deadlock, thrash, or ship a poisoned answer. This is why §6 insists on system-level evaluation as non-optional.

4. Emergent behavior and controllability

This is the fundamental tension of the whole autonomy spectrum, stated precisely: **as you increase the number of decisions a system makes without your review, the set of things it might do grows faster than your ability to enumerate them in advance**. A single LLM call has a small, auditable space of possible outputs. A multi-agent system running for an unbounded number of turns can, in principle, reach any state reachable by the combination of every agent's choices at every step — nobody designed that specific trajectory, it *emerged* from the interaction of simpler, individually-reasonable local decisions. That's not a bug in any one component; it's a property of the system.

This is precisely the tension that governance frameworks exist to manage (**Module 12 — AI Governance** [12-ai-governance]): governance isn't paperwork bolted onto agentic systems after the fact, it's the discipline of deciding, in advance, how much emergent behavior you're willing to tolerate for a given class of task, and what evidence you need before you trust the system to run further than you can watch.

Human-in-the-loop vs. human-on-the-loop

These are two different oversight postures, not two ends of a spectrum you slide along — pick per action, not per system:

Posture	Human's role	Latency cost	Use when
Human-in-the-loop (HITL)	Approves <i>before</i> an action executes; the system blocks and waits	High — every gated action pauses the pipeline	The action is irreversible, expensive, or externally visible: sending money, sending an email to a customer, deleting production data, merging code
Human-on-the-loop (HOTL)	Monitors a dashboard/log stream and can intervene, but the system proceeds by default	Low — the system doesn't wait for anyone	The action is reversible, cheap to undo, or low-stakes individually but worth aggregate monitoring: routine ticket triage, internal draft generation, read-only research

Concrete trigger conditions, not vibes:

- **Escalate to HITL when:** the action crosses a monetary threshold you set explicitly (e.g., refund over \$50), touches a system with no undo (production DB write, outbound email/SMS to an end user), or the agent's own confidence/self-reported uncertainty is below a threshold.
- **Stay at HOTL, but page a human when:** error rate over a rolling window exceeds a set percentage, a budget governor (§5) trips, or the same coordination failure mode (§3) recurs more than N times in a session — a symptom the topology itself can't self-correct.

The mistake to avoid: defaulting every action in a system to the *same* posture. A refund agent plausibly needs HITL on the "issue refund" tool call and HOTL on the "look up account" tool call. Gate by action, not by agent.

5. Long-running / autonomous agents

The far end of the spectrum — a computer-use agent completing a multi-app workflow overnight, a coding agent running for hours, a research agent given a week — introduces failure modes that don't show up in a five-minute session.

- **Checkpointing and resumability (concept-level).** A long-running agent must be able to persist its state (what it has done, what it has learned, what it still intends to do) at intervals, so a crash, a timeout, or a deliberate pause doesn't lose hours of work and doesn't force a costly re-derivation from scratch. Conceptually this is the same idea as a database transaction log or a training checkpoint (Module 02): capture enough state that you can resume from *here* rather than from zero. The concrete mechanics — thread IDs, checkpointers, the Store vs. checkpoint distinction — live in Module 16's LangGraph track

[16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md]; this module cares only that the property must exist, by design, before you let anything run unattended.

- **Goal drift over long horizons.** The longer an agent runs without a check against its original objective, the more its intermediate decisions can compound into an outcome that technically follows from each local choice but no longer serves the actual goal — the agentic-systems version of "the map is not the territory" repeated a hundred times until the map has wandered off the territory entirely. A coding agent asked to "fix the failing test" that instead spends six hours refactoring the surrounding module is exhibiting goal drift: each individual step looked justified, and the sum stopped being the task. The mitigation is periodic re-grounding: force the agent (or a supervisor watching it) to re-state the original goal and check current progress against it, not just against its own most recent sub-goal.
- **Budget/resource governors — a required safety mechanism, not an optimization.** A hard ceiling on tokens, tool calls, wall-clock time, or dollars spent, enforced *outside* the agent's own judgment, because an agent that has drifted or is stuck in a loop cannot be trusted to notice and stop itself. Treat this the same way you'd treat a circuit breaker: you hope it never trips, but the system is unsafe to run without one. This is not the same as cost optimization (choosing a cheaper model) — it's a hard stop that exists purely so a failure mode has a ceiling.

6. Evaluating agentic systems

Everything in §3 is invisible to a naive eval. A rigorous eval for an agentic system has to check more than "did each agent do its job":

- **System-level task success vs. per-agent success.** Measure whether the *system* achieved the actual goal, not whether each agent completed its assigned sub-task — a system can be 100% per-agent-successful and still produce a wrong final answer if a hand-off dropped the one detail that mattered (`code/handoff_information_loss.py` is this exact scenario, made measurable). Report both numbers; the gap between them is diagnostic.
- **Simulation environments and sandboxed evals.** Because agentic systems act (send emails, write files, call APIs), evaluating them against production is unacceptable. You need a sandbox that mimics the real environment closely enough that success there predicts success in production — a fake filesystem, a mock payment API, a scripted user simulator for multi-turn conversations. The fidelity of the sandbox is itself a variable you have to justify, the same way a simulator's fidelity matters in robotics.
- **Multi-agent eval is a research-open problem.** There is no agreed-upon standard for attributing a failure to the right agent, for scoring an emergent behavior nobody explicitly specified, or for reproducing a failure that depended on non-deterministic ordering between agents. Treat any vendor claim of a definitive "multi-agent benchmark" with real skepticism — the field genuinely hasn't solved this yet. Module 15 — AI Evaluation [15-ai-evals] covers the eval methodology in depth (LLM-as-judge, offline vs. online evals, regression suites); the multi-agent-specific addition is that your eval harness needs to capture the *trace* (who said what to whom, in what order) not just the final output, or you can't diagnose which of the five failure modes in §3 actually happened.

7. A decision framework: single-agent, multi-agent, or deterministic pipeline?

Work through these questions **in order**. Stop at the first "yes."

1. Can this be done in ONE bounded LLM call (classify, extract, summarize, one decision)?
YES -> single LLM call. Stop. You do not need an agent.
NO -> continue
2. Is the sequence of steps and their order KNOWN and fixed in advance, with only content (not control flow) varying at runtime?
YES -> deterministic pipeline with an LLM step at each stage where judgment is needed.
(This is almost always cheaper, more testable, and more debuggable than an agent.)
Stop.
NO -> continue
3. Does the task need a variable, not-knowable-in-advance number of steps, where each step's choice depends on what the previous step returned?
YES -> a single tool-using agent (Module 06). Stop, unless one of the conditions in step 4 is ALSO true.
NO -> re-examine step 1; you likely over-scoped the problem.
4. Does at least ONE of these hold?
 - sub-tasks are genuinely independent and parallelizable
 - different sub-tasks need different permissions / data residency
 - the tool list is too long for one model to select from reliably
 - sub-tasks genuinely warrant different models / cost tiersYES -> multi-agent system. Pick the topology from notes/01 whose "problem shape" column matches, budget for the ~4-15x token multiplier, and design the hand-off schema BEFORE writing agent code.
NO -> stay at step 3's single agent. Splitting now buys you cost and complexity with no offsetting benefit.

The checklist is deliberately biased toward the left (less autonomy, less machinery). That bias is the point: every step you don't need to take right is a step you don't pay for, don't have to debug, and don't have to explain to an auditor in Module 12 [12-ai-governance] or an eval harness in Module 15 [15-ai-evals].

8. Forward / backward links

Idea here	Where it connects
Context rot inside one agent's window	Module 04 — LLMs [04-llm]
Self-consistency / ensembling as debate's ancestor	Module 01 — ML Foundations [01-ml-foundations], Module 04 — sampling strategies [04-llm/code/sampling_strategies.py]
MCP — agent to tool/data	Module 09 — MCP [09-mcp]
These topologies as graph subgraphs, checkpointer mechanics	Module 16 — LangGraph track [16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md]
Governance response to emergent behavior	Module 12 — AI Governance [12-ai-governance]
Reference architectures that embed a chosen topology	Module 10 — AI Architecture [10-ai-architecture]
LLM-as-judge, regression suites, online evals	Module 15 — AI Evaluation [15-ai-evals]
Prompt injection propagating across agent hand-offs	Module 13 — AI Security [13-ai-security]